HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
School of Information and Communication Technology

Computer Vision Capstone Report

Evaluating Modular Representation and Recognizer Pipelines for ASL Alphabet Recognition

Does a Smarter Representation Actually Help, or Just Add a New Way to Fail?

Students: Tran Quang Hung – 20235502
Do Dang Vu – 20235578
Le Hoang Tung – 20235572

Topic: Computer Vision – Sign Language Recognition
Architecture: Modular Representation × Recognizer framework
Backbones: ResNet-18, SigLIP ViT, MLP, SVM, Random Forest
Dataset: ASL Alphabet (Kaggle)

June 21, 2026

Contents

Abstract 1

1 Introduction 1
- 1.1 Why Automatic Sign Language Recognition Matters . . . 1
- 1.2 Project Motivation: Does a Smarter Representation Help? . . . 2
- 1.3 Report Scope . . . 2
- 1.4 Main Contributions of This Report . . . 3

2 Problem Formulation 3

3 Proposed Framework 3
- 3.1 Two-Module Architecture . . . 3
- 3.2 Representation Functions . . . 4
- 3.3 Recognizers . . . 5

4 Evaluation Protocol 5
- 4.1 The Hidden Accuracy Pitfall . . . 5
- 4.2 Robustness Benchmark . . . 5

5 Experimental Setup 5
- 5.1 Dataset . . . 5
- 5.2 Why Clean Accuracy Alone Is Insufficient . . . 6

6 Quantitative Results 6

7 Qualitative Results 7
- 7.1 Step-by-Step Pipeline Behavior . . . 7
- 7.2 Looking Inside the CNN: Grad-CAM . . . 8
- 7.3 Feature-Space Analysis . . . 8

8 Case Study: High Reported Accuracy, Yet the System Still Fails 9

9 Discussion 10

10 Limitations 11

11 Conclusion 11

12 References 11

Abstract

This report studies whether "smarter" visual representations actually improve static American Sign Language (ASL) alphabet recognition, or merely add complexity and new failure modes. We build a modular framework that decouples a *Representation* module (raw image, MediaPipe hand-crop, MediaPipe landmarks, or image enhancement) from a *Recognizer* module (ResNet-18, a pretrained SigLIP Vision Transformer, or a landmark classifier — MLP, SVM, or Random Forest), yielding 13 distinct, independently evaluable pipelines. Beyond clean-image accuracy, we evaluate macro-F1, robustness under eight image corruptions, and a previously-hidden metric — the fraction of images for which the upstream hand detector fails and is silently excluded from the standard accuracy computation. We further use Grad-CAM and UMAP/t-SNE projections of the recognizer's feature space to inspect *why* pipelines succeed or fail, not just by how much. The results show that pipelines built on MediaPipe hand detection report competitive clean accuracy (up to 98.5%) yet collapse to 0% accuracy under the worst image corruption, and that the true end-to-end accuracy of the best such pipeline is 81.15%, not the 98.5% it reports — a 17-point gap hidden by the standard metric. Simpler pipelines that skip hand detection entirely (raw image or lightweight enhancement, fed directly to ResNet-18 or SigLIP) never drop below 70% accuracy under any corruption, showing that architectural simplicity can be a robustness advantage, not just a baseline.

1. Introduction

1.1 Why Automatic Sign Language Recognition Matters

American Sign Language (ASL) is the primary language of an estimated half a million to two million Deaf and hard-of-hearing people in North America, and manual fingerspelling of the alphabet is used whenever a signer needs to produce a word that has no dedicated sign — names, addresses, technical terms, or borrowed vocabulary. The communication gap this creates is not abstract: a Deaf person interacting with a hearing person who does not know ASL has no shared channel, and the burden of bridging that gap — hiring an interpreter, exchanging written notes, or relying on lip-reading — falls disproportionately on the Deaf individual. Figure 1 summarizes the causal chain from this communication gap to the system studied in this report.

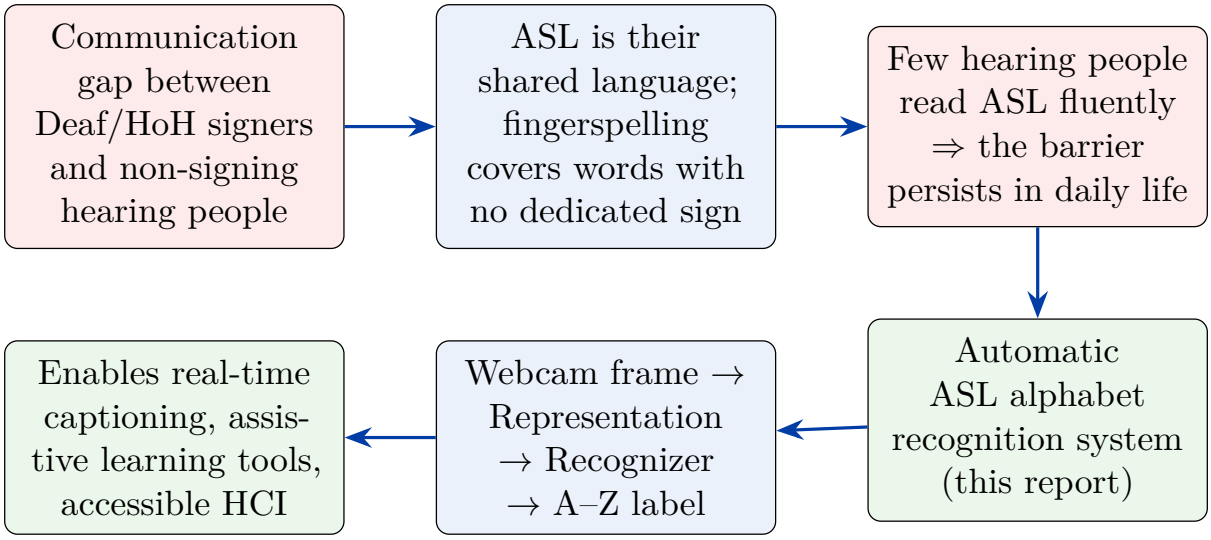


Figure 1: Motivation chain: a persistent communication gap (top row) motivates automatic ASL recognition; the system itself is the pipeline studied in this report (Section 2, bottom row, read right to left); its purpose is to enable downstream accessibility applications that do not require specialized hardware.

Building such a system only requires a standard webcam, which makes it substantially cheaper to deploy than dedicated sign-language gloves or depth-sensing hardware. The remaining question is purely a computer vision one: *which combination of preprocessing and classification produces a system that is accurate, robust, and fast enough to be useful* — and that question turns out to be far less settled than it first appears, because the intuitive idea that "adding a hand detector should help" is often false in practice, as Section 8 demonstrates directly.

1.2 Project Motivation: Does a Smarter Representation Help?

A static ASL alphabet recognizer is conventionally built as a two-stage pipeline: a *representation* stage that turns a raw webcam frame into something more tractable (e.g., a hand crop or hand-pose landmarks), followed by a *recognizer* stage that turns that representation into one of 26 letter labels. The representation stage is usually treated as an unambiguous improvement — cropping the hand removes background clutter, and landmarks remove skin-tone and lighting variation entirely. This project tests that assumption directly, asking:

- Does hand-cropping or landmark extraction actually improve recognition accuracy, or does it just add an additional component that can fail?
- Is a pretrained Vision Transformer worth its extra latency over a small, fine-tuned CNN?
- Does the standard way of reporting accuracy correctly reflect how the system would behave in real, uncontrolled use?
- Which pipeline is actually the best choice once accuracy, robustness, and speed are all considered together?

1.3 Report Scope

This report covers a single dataset (the Kaggle ASL Alphabet dataset, 26 static A–Z classes) and 13 distinct representation/recognizer combinations, evaluated under one shared protocol: clean accuracy, macro-F1, robustness under eight corruption types, latency, and (for MediaPipe-based

pipelines) hand-detection failure rate. Continuous, motion-based signs (e.g., J and Z in real ASL) are treated as static targets, consistent with the rest of the literature on static fingerspelling recognition.

1.4 Main Contributions of This Report

1. A modular framework that decouples representation and recognizer choices, enabling 13 pipelines to be compared under one protocol instead of 13 separate, inconsistent setups.
2. An evaluation protocol that exposes a metric pitfall hidden in the common practice of excluding "no hand detected" frames from the accuracy denominator (Section 4).
3. A robustness benchmark across eight corruption types that differentiates pipelines that look identical on clean data.
4. Qualitative tooling — step-by-step visual grids, Grad-CAM, and UMAP/t-SNE projections of the recognizer's feature space — that explains *why*, not just *whether*, a pipeline fails.
5. A concrete case study quantifying the gap between reported and real accuracy for the best-looking MediaPipe-based pipeline.

2. Problem Formulation

Let $x \in \mathcal{X}$ be an RGB webcam frame and $y \in \mathcal{Y} = \{A, \ldots, Z\}$ its ground-truth label. We decompose the recognition system into two independently swappable modules:

$$\hat{y} = R(\,\Phi(x)\,), \tag{1}$$

where $\Phi : \mathcal{X} \to \mathcal{Z}$ is the *representation function* and $R : \mathcal{Z} \to \mathcal{Y} \cup \{\textsc{Unknown}\}$ is the *recognizer*. Critically, Φ is *partial*: for MediaPipe-based representations, $\Phi(x)$ is undefined (returns $\varnothing$) whenever the upstream hand detector fails to locate a hand in x, in which case R trivially outputs UNKNOWN without ever running its classification logic. This single design fact is the source of the central finding of this report (Section 8): a pipeline's measured accuracy depends entirely on how images with $\Phi(x) = \varnothing$ are counted, and the conventional choice (excluding them) systematically overstates real-world performance.

A pipeline is therefore fully specified by the pair (Φ, R). We study 4 representation functions and 5 recognizers (Section 3), yielding 13 type-compatible (Φ, R) combinations (image-typed representations pair with image-based recognizers; the landmark representation pairs with landmark-based recognizers).

3. Proposed Framework

3.1 Two-Module Architecture

Figure 2 previews the four representation functions studied, applied to the same five test images; Table 1 lists the five recognizers. Any image-typed representation can be paired with any image-based recognizer, and the landmark representation pairs with any of the three landmark-based recognizers, for $4 \times 2 + 1 \times 3 = 11$ structurally distinct combinations; counting the two enhancement variants (CLAHE, Gamma) each paired with both image recognizers brings the evaluated set to 13 pipelines in total (Table 2).

3.2 Representation Functions

- **Raw image:** resize to 224×224; no other processing.
- **MediaPipe Crop:** MediaPipe HandLandmarker locates 21 hand keypoints, from which a bounding box is computed; the frame is cropped to that box (with a fixed pixel padding) and resized.
- **MediaPipe Landmarks:** the same 21 keypoints are kept as raw coordinates and converted into a 42-dimensional feature vector, normalized relative to the wrist keypoint and scaled by the maximum absolute coordinate value:

$$\tilde{p}_i = p_i - p_{\text{wrist}}, \qquad f_i = \frac{\tilde{p}_i}{\max_j |\tilde{p}_j|}, \qquad i = 1, \ldots, 21, \tag{2}$$

  producing $f \in \mathbb{R}^{42}$ (concatenated x, y per point) that is invariant to translation and scale but discards all texture and skin-color information.
- **Enhancement:** CLAHE, gamma correction, unsharp-mask sharpening, or non-local-means denoising applied to the raw image before resizing, used to test whether photometric preprocessing (rather than spatial cropping) helps.

Figure 2: Five sample test images (rows) through six concrete representation instances (columns): Raw, MediaPipe Crop, CLAHE, Gamma, Sharpening, Denoising. The MediaPipe Crop column is black on 3 of 5 rows — the hand detector failed on those (low-contrast) images, illustrating the partial nature of Φ described in Section 2.

3.3 Recognizers

Table 1: Recognizer modules and their expected input type.

Recognizer	Model family	Input
ResNet-18 (`resnet18_asl` / `torchvision_classifier`)	CNN, fine-tuned on ASL	224×224 image
SigLIP ViT (HuggingFace)	Vision Transformer, pretrained	224×224 image
Landmark MLP	Multi-layer perceptron	42-d vector
Landmark SVM	Support Vector Machine	42-d vector
Landmark RF	Random Forest	42-d vector

The pretrained ViT recognizer is the strongest model in isolation; the landmark-based classifiers are the lightest and fastest (Table 2). The central question of this report is whether pairing a strong recognizer with a hand-cropping representation actually compounds into a better *system*, or whether the representation stage's own failure rate cancels out the recognizer's quality gain.

4. Evaluation Protocol

4.1 The Hidden Accuracy Pitfall

The implementation's standard accuracy metric is computed only over the subset of test images for which the representation successfully produced an output:

$$\text{clean\_accuracy} = \frac{\#\{i : \hat{y}_i = y_i \wedge \Phi(x_i) \neq \varnothing\}}{\#\{i : \Phi(x_i) \neq \varnothing\}}. \tag{3}$$

Images with $\Phi(x_i) = \varnothing$ (no hand detected) are removed from *both* the numerator and the denominator, rather than counted as incorrect. We additionally compute the metric that matches how a deployed system would actually be judged by a user:

$$\text{real\_accuracy} = \frac{\#\{i : \hat{y}_i = y_i\}}{\#\{i : 1 \leq i \leq N\}}, \qquad \text{where } \hat{y}_i = \textsc{Unknown} \text{ counts as incorrect.} \tag{4}$$

We also report the *hand-detection failure rate*, $\text{HDFR} = \#\{i : \Phi(x_i) = \varnothing\}/N$, which applies only to MediaPipe-based representations (it is identically zero for Raw and Enhancement, since those representations never fail to produce an output). Section 8 shows that the gap between Eq. (3) and Eq. (4) can exceed 17 percentage points for a pipeline with a high HDFR.

4.2 Robustness Benchmark

To test behavior beyond the (visually clean, studio-lit) test set, every pipeline is re-evaluated on eight corrupted variants of the test set: low-light, overexposure, motion blur, Gaussian noise, rotation, scale shift, partial occlusion, and crop shift. We report the worst single-corruption accuracy (the pipeline's failure ceiling) and the average accuracy drop relative to clean accuracy.

5. Experimental Setup

5.1 Dataset

We use the Kaggle ASL Alphabet dataset [4]: 26 static A–Z classes, photographed under relatively uniform studio lighting against a fairly clean background. The training split (∼78,000

images) is used to fine-tune ResNet-18 and to train the landmark MLP/SVM/RF classifiers; the `test_split` subset (2,600 images, 100 per class) is held out for all clean-accuracy and robustness evaluation reported in this paper. The ViT recognizer uses third-party pretrained weights and is not further fine-tuned on this dataset.

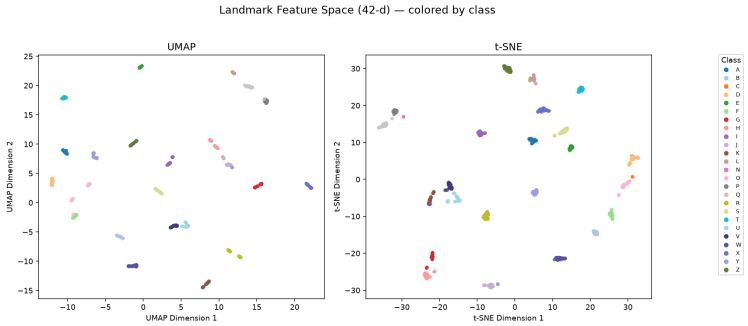


Figure 3: UMAP (left) and t-SNE (right) projection of the 42-dimensional MediaPipe landmark vectors (Eq. 2), 30 images per class sampled from `test_split`, colored by ground-truth class. All 26 classes form well-separated, compact clusters purely from hand pose — the dataset is intrinsically easy to classify when the hand is successfully detected, which is exactly why clean accuracy alone fails to distinguish pipeline quality (Section 8): nearly every pipeline that sees the hand classifies it correctly.

5.2 Why Clean Accuracy Alone Is Insufficient

The dataset's studio conditions, combined with the clean class separability shown in Figure 3, mean that most pipelines reach 99–100% clean accuracy (Table 2). This is precisely the scenario the robustness benchmark is designed for: when nearly every pipeline looks equally good on clean data, only stress-testing under corruption and auditing the accuracy formula itself (Section 4) reveal meaningful differences.

6. Quantitative Results

Table 2 reports clean accuracy, macro-F1, worst-case corruption accuracy, hand-detection failure rate, and inference speed for all 13 pipelines, sorted by clean accuracy.

Table 2: Full results across 13 pipelines on the ASL Alphabet `test_split`. Bold rows are referenced directly in Section 8.

Pipeline	Clean Acc.	Macro-F1	Worst-case	Hand-fail	FPS
raw_siglip	1.0000	1.0000	0.8912	0.0%	6.3
enhancement_gamma_vit	1.0000	1.0000	0.8927	0.0%	22.9
enhancement_clahe_vit	0.9996	0.9996	0.8629	0.0%	9.3
raw_resnet18	0.9985	0.9985	0.9202	0.0%	222.3
enhancement_gamma_resnet18	0.9985	0.9985	0.9135	0.0%	21.8
enhancement_sharpen_resnet18	0.9965	0.9965	¬0.1010	0.0%	244.6
enhancement_clahe_resnet18	0.9954	0.9954	0.7356	0.0%	16.7
enhancement_denoise_resnet18	0.9954	0.9954	0.7012	0.0%	7.9
mediapipe_crop_vit	**0.9851**	0.9707	¬0.0000	**17.15%**	11.2
mediapipe_landmarks_rf	0.9741	0.9552	¬0.0000	17.15%	16.6
mediapipe_landmarks_svm	0.9706	0.9723	¬0.0000	17.15%	19.8
mediapipe_landmarks_mlp	0.9705	0.9556	¬0.0000	17.15%	35.0
mediapipe_crop_resnet18	**0.8859**	0.8627	¬0.0000	**17.15%**	18.9

Three patterns are immediately visible. First, every pipeline that skips hand detection entirely (Raw or Enhancement representations) reaches at least 99.5% clean accuracy and never drops below 70% worst-case accuracy. Second, every pipeline built on MediaPipe (crop or landmarks) drops to exactly $\neg 0.0000 worst-case accuracy, regardless of how strong its recognizer is --- the SigLIP-based$ `mediapipe_crop_vit` $collapses just as completely as the much weaker$ `mediapipe_crop_resnet18`. $The detection failure rate is identical across all five MediaPipe-based pipelines, since it is a property of the shared repr$

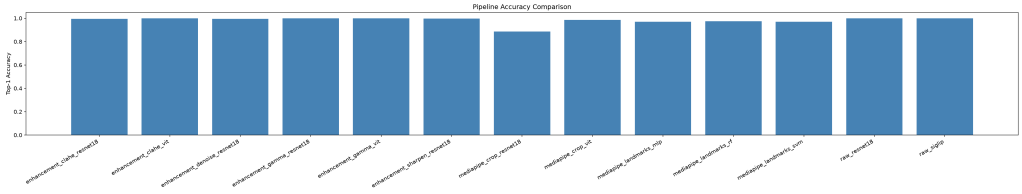


Figure 4: Clean accuracy across all 13 pipelines.

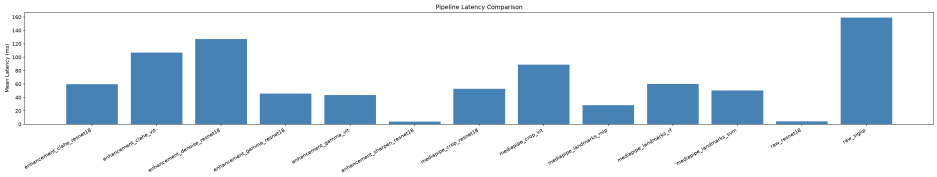


Figure 5: Latency / FPS trade-off. Landmark-based and raw-ResNet-18 pipelines reach the highest throughput, making the accuracy–robustness–speed trade-off (rather than accuracy alone) the right way to choose a pipeline for a given deployment target.

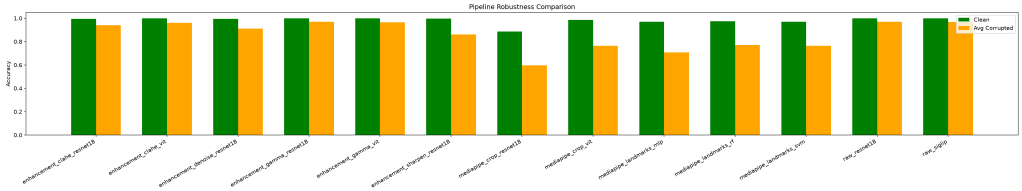


Figure 6: Accuracy under each of the eight corruption types, per pipeline. The MediaPipe-based pipelines are the only ones that reach zero accuracy on any corruption type.

7. Qualitative Results

7.1 Step-by-Step Pipeline Behavior

Figure 7 extends Figure 2 to full pipelines: the same five images are passed through all 13 pipelines, and each prediction is marked correct (green) or incorrect (red).

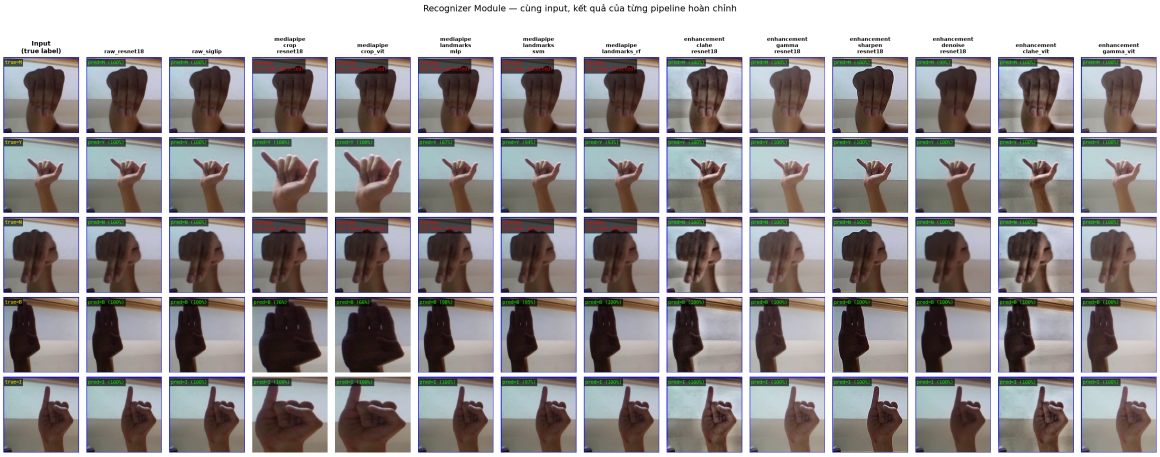
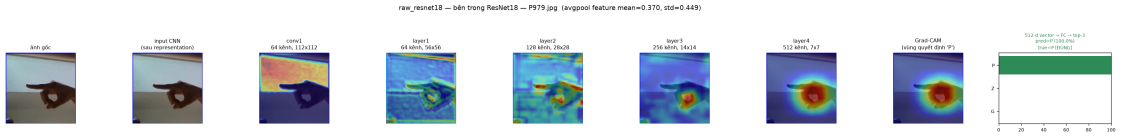


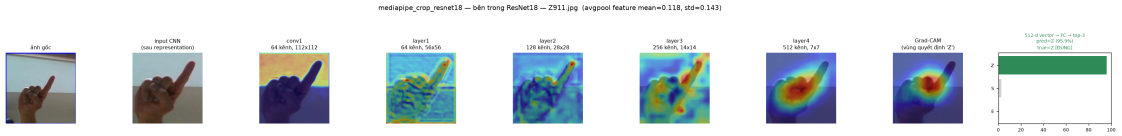
Figure 7: Predictions of all 13 pipelines on the same five test images. Every MediaPipe-based pipeline (columns sharing the MediaPipe representation) is wrong on the exact same rows — the failure originates once in the shared representation stage and propagates identically to every recognizer attached to it.

7.2 Looking Inside the CNN: Grad-CAM

To confirm that the ResNet-18 classifier itself is not the source of failure, Figure 8 visualizes Grad-CAM activations at `layer4` for two pipelines that share the same recognizer weights but differ in representation.



(a) `raw_resnet18`: activation maps shrink from 224^2 (conv1) to 7^2 (layer4) while channel count grows $64 \rightarrow 512$; Grad-CAM correctly highlights the hand region.



(b) `mediapipe_crop_resnet18`: same backbone, cropped input; Grad-CAM again correctly attends to the hand.

Figure 8: Grad-CAM at the final convolutional block for two pipelines sharing ResNet-18 weights. In both cases the classifier attends to the correct region — confirming that classification failures observed elsewhere in this report originate in the upstream representation stage, not in the CNN itself.

7.3 Feature-Space Analysis

Figure 9 confirms that ResNet-18's learned 512-d feature space separates classes just as cleanly as the raw 42-d landmark space (Figure 3), reinforcing that classification itself is not the bottleneck for any pipeline studied. Figure 10 then asks a different question: does *changing the representation* (rather than the class) move a fixed image's feature vector?

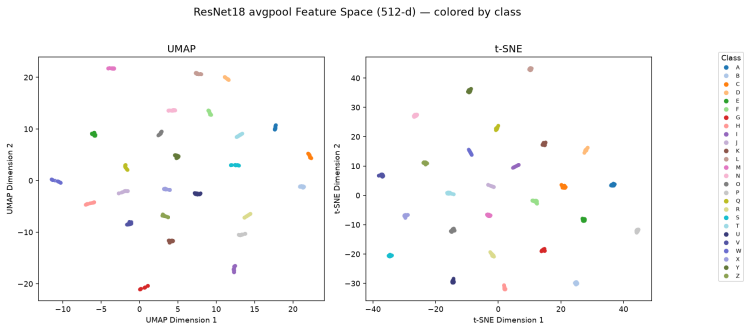


Figure 9: UMAP/t-SNE of 512-d ResNet-18 `avgpool` embeddings (30 images/class). Classes separate as cleanly as in landmark space (Figure 3).

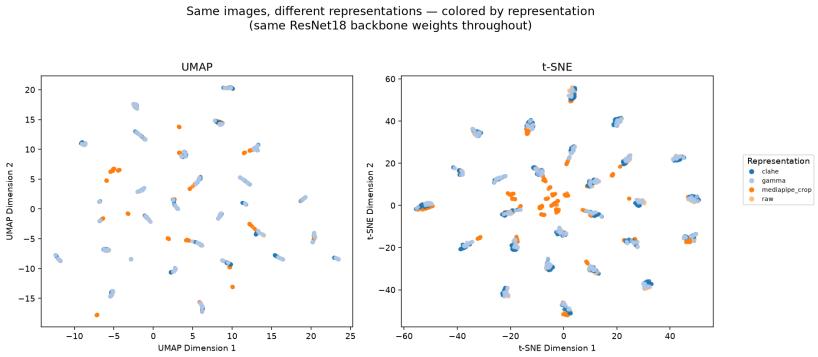


Figure 10: Same images, four representations (Raw, MediaPipe Crop, CLAHE, Gamma), embedded through the same ResNet-18 backbone and colored by representation. Raw/CLAHE/Gamma cluster together per image — mild photometric enhancement barely moves the feature vector — while MediaPipe Crop (orange) lands in a visibly different region: changing the *representation*, not just lightly enhancing the same image, is what actually shifts the feature space the recognizer operates in.

8. Case Study: High Reported Accuracy, Yet the System Still Fails

This section makes the abstract metric pitfall of Section 4 concrete using `mediapipe_crop_vit` — a pipeline that combines MediaPipe hand-cropping with the strongest recognizer in this study, a pretrained SigLIP ViT.

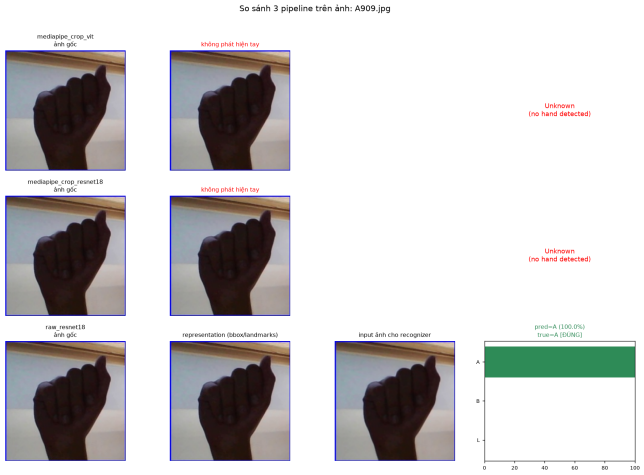


Figure 11: The same low-contrast test image, run through three pipelines. `mediapipe_crop_vit` and `mediapipe_crop_resnet18` both output UNKNOWN — the upstream hand detector failed before either recognizer had a chance to classify anything. `raw_resnet18`, which has no detection stage, correctly predicts "A" with 100% confidence on the identical pixels.

Applying Eq. (3), `mediapipe_crop_vit` reports a clean accuracy of 98.5%, the best result among MediaPipe-based pipelines (Table 2). But this number is computed only over the $N - HDFR \cdot N \approx 2{,}142$ images on which a hand was detected. Applying Eq. (4) over the full 2,600-image test set:

$$\text{real\_accuracy(mediapipe\_crop\_vit)} = \frac{2110}{2600} = 81.15\%, \tag{5}$$

a gap of **17.4 percentage points** below the reported figure. The recognizer itself is not at fault — as Figure 8 confirms, the classifier correctly attends to hand regions when it receives them; the gap exists purely because 17.15% of inputs never reach the classifier at all, and the standard accuracy formula does not count that as failure. The practical conclusion is unambiguous: a two-stage pipeline's reliability is upper-bounded by its weakest stage, and that bound is invisible unless accuracy is computed over *all* inputs, not only the ones a detector chose to act on.

9. Discussion

Three findings, taken together, answer the question posed in Section 1.1. First, hand-cropping and landmark extraction do not improve the recognizer's job in this dataset — Figure 3 shows the classes are already linearly separable from raw landmarks, and Figure 9 shows ResNet-18 separates them just as well from raw pixels, so there is no classification difficulty for a smarter representation to solve. Second, the representation stage introduces a new failure mode (missed hand detection) that the recognizer cannot recover from, and this failure mode is invisible in the standard accuracy metric (Section 8). Third, the failure compounds catastrophically, not gracefully, under corruption (Figure 6): rather than a modest accuracy drop, MediaPipe-based pipelines reach exactly 0% accuracy on at least one corruption type, because a single brittle upstream stage gates the entire pipeline.

The practical implication for system design is that architectural simplicity should be the default, not the fallback: a raw-image or lightly-enhanced representation fed directly to a single classifier is both more accurate end to end and structurally immune to this class of failure, since there is no detection stage to fail in the first place.

10. Limitations

Several representation modules described in the project's design (YOLO-based cropping, segmentation-mask cropping, and RGB+landmark fusion) remain unimplemented stubs and are not part of the 13 evaluated pipelines. All results are reported on a single dataset (ASL Alphabet, studio-lit, relatively clean backgrounds); the absolute hand-detection failure rate and the size of the accuracy gap in Section 8 would likely be different, and plausibly larger, on uncontrolled real-world webcam footage. The SigLIP ViT recognizer is used as a third-party pretrained model without further fine-tuning on this dataset, so its reported numbers reflect transfer performance rather than a fully optimized model.

11. Conclusion

We built a modular framework that separates the representation and recognizer stages of a static ASL alphabet recognition system, and used it to evaluate 13 pipelines under one shared protocol covering clean accuracy, robustness, and a previously-hidden accuracy pitfall. The results show that adding a hand detector does not improve recognition in this setting: it does not make classification easier (the classes are already separable without it), and it introduces a brittle stage that silently removes 17% of inputs from the standard accuracy calculation while contributing zero accuracy under several corruption types. The simplest pipelines studied — a raw or lightly enhanced image fed directly to a single recognizer — are simultaneously the most accurate end to end and the most robust, directly answering the motivating question of this report: a smarter-looking representation is not automatically a better one, and verifying that claim requires evaluating the full pipeline under metrics that cannot hide its failures.

12. References

References

[1] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," *arXiv:1906.08172*, 2019.

[2] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," *IEEE Conf. on Computer Vision and Pattern Recognition (CVPR)*, 2016.

[3] X. Zhai, B. Mustafa, A. Kolesnikov, and L. Beyer, "Sigmoid Loss for Language Image Pre-Training," *IEEE Int. Conf. on Computer Vision (ICCV)*, 2023.

[4] Kaggle, "ASL Alphabet Dataset," https://www.kaggle.com/datasets/grassknoted/asl-alphabet.

[5] L. McInnes, J. Healy, and J. Melville, "UMAP: Uniform Manifold Approximation and Projection for Dimension Reduction," *arXiv:1802.03426*, 2018.

[6] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, 2008.

[7] R. R. Selvaraju et al., "Grad-CAM: Visual Explanations from Deep Networks via Gradient-based Localization," *IEEE Int. Conf. on Computer Vision (ICCV)*, 2017.